

Re: Initial Post

by Pëllumb Dalipi - Tuesday, 2 September 2025, 7:08 PM

I thank my peer Mansour for their clear overview. I agree that ACLs such as KQML give a shared semantic layer and performatives which reduce ambiguity across heterogeneous platforms (Labrou and Finin 1998).

Empirical work also notes cost in practice. Interoperation and ontology alignment remain difficult, which can raise parser effort and run time cost (Mayfield, Labrou and Finin 1996; Koné, Shimazu and Nakajima 2000). Security is not native in core FIPA ACL, which led to separate security specifications and platform add ons such as JADE S (Poslad, Buckle and Hadingham 2003; Bellifemine, Caire and Greenwood 2001). Direct method calls in Java or Python give speed and clear traces, yet they assume a shared runtime and common types, so suitability drops in open systems (Wooldridge 2009). A constructive path uses social or commitment based protocol design, which keeps semantic intent and yields clearer contracts for engineers while it tempers cost (Singh 1998). Choice should follow scale and heterogeneity. ACLs fit open, varied networks. Direct calls fit unified environments.

References

- Bellifemine F, Caire G and Greenwood D 2001. JADE A FIPA compliant agent framework. In: Proceedings of the International Conference on Autonomous Agents.
- Koné M T, Shimazu A and Nakajima T 2000. The state of the art in agent communication languages. Knowledge and Information Systems.
- Labrou Y and Finin T 1998. Semantics and Conversations for an Agent Communication Language.
- Mayfield J, Labrou Y and Finin T 1996. Evaluation of KQML as an Agent Communication Language. In: Wooldridge M, Müller J P and Tambe M eds. Intelligent Agents II.
Poslad S, Buckle P and Hadingham R 2003. Specifying standard security mechanisms in multi agent systems. In: Intelligent Agents VIII.
Singh M P 1998. Agent Communication Languages Rethinking the Principles. IEEE Computer.
Wooldridge M 2009. An Introduction to Multiagent Systems.

Peer Response

by Fahad Abdallah - Saturday, 6 September 2025, 7:34 AM

Hi Mansour, you have presented a clear and well-organised description of the concept of Agent Communication Languages (ACLs) or KQML and how the languages of communication enable agents to interact successfully with other agents in multi-agent systems. Your focus on standardisation and interoperability is critical since MAS must manage heterogeneous agents coordinating across different platforms and tasks. Recent literature emphasising the advantages of large language model-based agents to take advantage of structured communication structures to maximise coordination in dynamic and complex environments (Cheng et al., 2024) reinforces this view.

I concur with your argument that the ontological nature of KQML enhances teamwork in distributed settings. By conveying domain knowledge in an organised manner, agents are able to develop shared understanding and attain more advanced types of collaboration, which have been noted as a major characteristic in classic and more contemporary MAS studies (Zhang et al., 2024).

Verbosity, complexity, and security risks are also valid points you made. These constraints reflect existing issues in the literature, in which the implementation of knowledge-intensive communication must consume extensive computational resources and be carefully designed to be efficient and safe (Ma et al., 2024). Your analogy to method invocation in Python or Java is not invalid, since those tend to be more efficient in systems that are tightly coupled and are less able to offer the same semantic interoperability among agents of different kinds.

One potential continuation of your argument may be that the large language models (LLMs) are now embedded into agent communication protocols. Existing studies indicate that LLM-powered agents can exploit superior memory processes and natural language logic to address specific gaps of past ACL systems (Cheng et al., 2024; Zhang et al., 2024). This makes it possible to apply it practically outside of academia and could solve your scalability and adaptability problems.

References

Cheng, Y., Zhang, C., Zhang, Z., Meng, X., Hong, S., Li, W., ... & He, X. (2024). Exploring large language model-based intelligent agents: Definitions, methods, and prospects. *arXiv preprint arXiv:2401.03428*. Available at: <https://arxiv.org/pdf/2401.03428> (Accessed: 04 September 2025).

Ma, Z., Mei, Y., & Su, Z. (2024, January). Understanding the benefits and challenges of using large language model-based conversational agents for mental well-being support. In *AMIA Annual Symposium Proceedings* (Vol. 2023, p. 1105). Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10785945/pdf/406.pdf> (Accessed: 04 September 2025).

Zhang, Z., Dai, Q., Bo, X., Ma, C., Li, R., Chen, X., ... & Wen, J. R. (2024). A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*. Available at: <https://dl.acm.org/doi/pdf/10.1145/3748302> (Accessed: 04 September 2025).

Peer Response

by Rayyan Mohamed Abdalla Alshambeeli Alnaqbi - Monday, 8 September 2025, 5:00 PM
Hi Mansour,

You've done a great job of showing the main pros and cons of KQML and method invocation in agent communication. The focus on KQML's standardized performative verbs does make interactions between multiple agents clearer and less confusing, which is important for making sure that different systems can work together (Labrou et al., 1999). Your point about how the ontology-based framework makes it easier to work together in changing environments also shows why ACLs are still useful in complicated AI applications (Finin et al., 1997).

But, as you pointed out, KQML's long and complicated syntax can slow down computers and raise security issues, especially in open communication protocols (Huhns and Singh, 1998). This trade-off often makes it less useful in systems that aren't very big or very different from each other.

In contrast, calling a method in Python and Java is a lighter and more flexible way to do things, which is best for smaller or more integrated systems. The ability to quickly prototype and debug in these languages speeds up development cycles and makes it easier to put things into use (Lutz, 2013).

I agree with what you said about how the size and variety of a system affect the choice between ACLs and method invocation (Wooldridge, 2009). To design smart systems that meet both academic and real-world needs, you need to know about these trade-offs.

References:

- Labrou, Y., Finin, T. and Peng, Y. (1999) 'The current landscape of agent communication languages', *IEEE Intelligent Systems*, 14(2), pp. 45–52. [Online] Available at: <https://doi.org/10.1109/5254.758857> [Accessed 8 Sep. 2025].
- Huhns, M.N. and Singh, M.P. (1998) 'Agent-oriented software engineering: The state of the art', *IEEE Expert*, 13(6), pp. 72–79. [Online] Available at: <https://doi.org/10.1109/64.736466> [Accessed 8 Sep. 2025].